

# The Burp Suite Playbook

A complete, detailed guide to web application testing  
with Burp Suite — from intercepting a request to a full assessment

*From a single intercepted request to a scripted Intruder attack,  
this playbook builds the tool fluency and testing methodology that  
web application penetration testers rely on every day.*

**Author:** Swastik Sagar  
Final-year Computer Science Undergraduate  
SOC Analyst Intern

August 13, 2026

# Preface

Almost every web application security assessment runs through Burp Suite at some point — it sits as a proxy between browser and server, making every request and response visible and editable, and builds a full testing toolkit around that single vantage point. Understanding Burp well means understanding not just its individual tools, but how they compose into a testing workflow: intercept and map an application, then probe it methodically with Repeater and Intruder, then verify findings.

This playbook follows that workflow in order: proxy setup and traffic interception first, then the core manual testing tools (Repeater, Intruder, Decoder, Comparer), then Burp’s active/passive scanner, then extensions, then a chapter mapping Burp’s tools directly onto the OWASP Top 10 categories they’re most useful for testing. Every chapter includes the exact configuration and workflow needed to use the tool for real.

## Authorization required

Every technique in this book is intended strictly for authorized web application testing, bug bounty programs within their disclosed scope, and personal lab/CTF environments. Testing applications you don’t own or have explicit written authorization to test is illegal in most jurisdictions.

## What this book covers

- Burp’s architecture and how its tools share one traffic vantage point
- Proxy setup: browser configuration and the CA certificate
- Target and scope configuration
- Intercepting and modifying live traffic
- Repeater, for manual request manipulation and replay
- Intruder, all four attack types and payload configuration
- Decoder and Comparer
- Burp Scanner: active and passive scanning
- Extensions and the BApp Store
- Mapping Burp’s tools onto OWASP Top 10 testing
- Practical testing workflows
- Detection and defense against Burp-based testing

*Swastik Sagar*

# Contents

|                                                       |          |
|-------------------------------------------------------|----------|
| <b>Preface</b>                                        | <b>i</b> |
| <b>1 Burp Suite Architecture</b>                      | <b>1</b> |
| 1.1 What Burp Suite is . . . . .                      | 1        |
| 1.2 How traffic flows through Burp . . . . .          | 1        |
| 1.3 The core tools, at a glance . . . . .             | 1        |
| <b>2 Setting Up the Proxy</b>                         | <b>2</b> |
| 2.1 Configuring the browser . . . . .                 | 2        |
| 2.2 Installing Burp's CA certificate . . . . .        | 2        |
| 2.3 Verifying interception works . . . . .            | 2        |
| <b>3 Target and Scope</b>                             | <b>3</b> |
| 3.1 The Target tool . . . . .                         | 3        |
| 3.2 Defining scope . . . . .                          | 3        |
| 3.3 Site map annotations . . . . .                    | 3        |
| <b>4 Proxy: Intercepting and Modifying Traffic</b>    | <b>4</b> |
| 4.1 Intercept on vs. off . . . . .                    | 4        |
| 4.2 A typical intercept-and-modify workflow . . . . . | 4        |
| 4.3 Match and replace . . . . .                       | 4        |
| <b>5 Repeater</b>                                     | <b>5</b> |
| 5.1 What Repeater is for . . . . .                    | 5        |
| 5.2 Sending a request to Repeater . . . . .           | 5        |
| 5.3 A typical Repeater testing sequence . . . . .     | 5        |
| <b>6 Intruder</b>                                     | <b>6</b> |
| 6.1 The four attack types . . . . .                   | 6        |
| 6.2 Marking payload positions . . . . .               | 6        |
| 6.3 Payload types worth knowing . . . . .             | 6        |

|           |                                                                       |           |
|-----------|-----------------------------------------------------------------------|-----------|
| <b>7</b>  | <b>Decoder and Comparer</b>                                           | <b>8</b>  |
| 7.1       | Decoder . . . . .                                                     | 8         |
| 7.2       | Comparer . . . . .                                                    | 8         |
| <b>8</b>  | <b>Burp Scanner</b>                                                   | <b>9</b>  |
| 8.1       | Passive vs. active scanning . . . . .                                 | 9         |
| 8.2       | Running an active scan . . . . .                                      | 9         |
| 8.3       | Reviewing scan results . . . . .                                      | 9         |
| <b>9</b>  | <b>Extensions and the BApp Store</b>                                  | <b>10</b> |
| 9.1       | What extensions add . . . . .                                         | 10        |
| 9.2       | Notable extensions worth knowing . . . . .                            | 10        |
| <b>10</b> | <b>Testing the OWASP Top 10 with Burp</b>                             | <b>11</b> |
| 10.1      | Injection . . . . .                                                   | 11        |
| 10.2      | Broken authentication . . . . .                                       | 11        |
| 10.3      | Broken access control . . . . .                                       | 11        |
| 10.4      | Security misconfiguration . . . . .                                   | 11        |
| <b>11</b> | <b>Practical Testing Workflows</b>                                    | <b>12</b> |
| 11.1      | Workflow: testing a login form for SQL injection . . . . .            | 12        |
| 11.2      | Workflow: IDOR testing across two accounts . . . . .                  | 12        |
| 11.3      | Workflow: directory and parameter discovery . . . . .                 | 12        |
| <b>12</b> | <b>Detection and Defense</b>                                          | <b>13</b> |
| 12.1      | What Burp-based testing looks like from the defender's side . . . . . | 13        |
| 12.2      | Defensive measures . . . . .                                          | 13        |

# Burp Suite Architecture

## 1.1 What Burp Suite is

Burp Suite is an integrated platform for web application security testing, built around a core idea: position yourself as an intercepting proxy between the browser and the target application, and every other tool in the suite operates on traffic captured at that single point.

## 1.2 How traffic flows through Burp



**Figure 1.1.** *Every request and response passes through the proxy in both directions – this single vantage point is what every other Burp tool builds on.*

## 1.3 The core tools, at a glance

| Tool     | Purpose                                                                   |
|----------|---------------------------------------------------------------------------|
| Proxy    | Intercepts and lets you modify live traffic between browser and server    |
| Target   | Maps the application's structure as you browse it                         |
| Repeater | Manually resend and edit individual requests                              |
| Intruder | Automates sending many variations of a request                            |
| Decoder  | Encode/decode and hash data in common formats                             |
| Comparer | Diff two pieces of data (requests, responses, or arbitrary text)          |
| Scanner  | Automated active/passive vulnerability scanning (Burp Suite Professional) |

### Community vs. Professional

Burp Suite Community Edition includes Proxy, Repeater, Intruder (rate-limited), Decoder, and Comparer; the Scanner and unrestricted Intruder speed are Professional-only features — worth knowing since most walkthroughs online assume Professional's full feature set.

# Setting Up the Proxy

## 2.1 Configuring the browser

Burp's proxy listens locally (by default 127.0.0.1:8080) and the browser needs to be pointed at it — either through the OS/browser proxy settings directly, or more conveniently through the FoxyProxy extension for quick proxy on/off toggling.

### Default proxy listener

127.0.0.1:8080

## 2.2 Installing Burp's CA certificate

Since most traffic today is HTTPS, Burp needs to present its own CA certificate to the browser so it can transparently decrypt and re-encrypt traffic — without installing and trusting this certificate, every HTTPS site will show a certificate warning.

- Browse to `http://burp` while proxying through Burp
- Download the CA certificate from that page
- Import it into the browser's (or OS's) trusted certificate store

### Why this isn't a security risk in your own lab

This works because Burp is performing a deliberate, consented man-in-the-middle on your own traffic — the same mechanism that would be a genuine attack if done without consent is exactly what makes the proxy useful for testing your own or authorized applications.

## 2.3 Verifying interception works

Once configured, browsing to any site should show live requests appearing in *Proxy & HTTP history* — if nothing appears, double check the browser's proxy settings and that *Intercept is off* (so traffic passes through to history without blocking on each request).

## Target and Scope

### 3.1 The Target tool

As you browse an application through the proxy, Burp automatically builds a site map in the *Target* tab — every discovered URL, parameter, and form, organized into the application's actual directory structure.

### 3.2 Defining scope

- Right-click a host in the site map and choose *Add to scope*
- Configure *Target & Scope* directly with include/exclude rules
- Enable *Proxy & Options & "Show only in-scope items"* to declutter history

#### Scope discipline matters

Defining scope precisely isn't just for organization — Intruder attacks and Scanner runs against out-of-scope hosts are exactly the kind of mistake that turns an authorized test into an unauthorized one, so scope should be configured before any active testing begins.

### 3.3 Site map annotations

Burp lets you manually flag interesting requests in the site map with comments and highlight colors — a practical way to bookmark findings and areas worth deeper testing as you work through a large application without losing track between tools.

# Proxy: Intercepting and Modifying Traffic

## 4.1 Intercept on vs. off

With *Intercept* on, every request pauses in the Proxy tab before being forwarded, letting you edit it live before it reaches the server; with it off, traffic passes through automatically and is simply logged to *HTTP history* for later review.

## 4.2 A typical intercept-and-modify workflow

### The general pattern

1. Turn Intercept on
2. Perform the action in the browser (e.g. submit a login form)
3. The request pauses in Burp -- edit any header, parameter, or body value
4. Click Forward to send the modified request
5. Observe the server's response

## 4.3 Match and replace

*Proxy > Options > Match and Replace* automatically rewrites matching content in every request or response passing through — useful for persistent changes like swapping a User-Agent, injecting a header on every request, or removing a security header for testing purposes, without editing each request by hand.

### Intercepting is slow for browsing, useful for testing

Leaving Intercept on while simply browsing an application makes it nearly unusable — the normal workflow is browsing with Intercept off to build up history, then switching individual requests to Repeater for the actual manual testing.

# Repeater

## 5.1 What Repeater is for

Repeater takes a single captured request and lets you edit and resend it repeatedly, comparing responses side by side — the core tool for manually probing exactly how an application responds to a specific crafted input.

## 5.2 Sending a request to Repeater

From anywhere in Burp

Right-click any request -> "Send to Repeater"  
Keyboard shortcut: Ctrl+R (Cmd+R on macOS)

## 5.3 A typical Repeater testing sequence

- Send a baseline request first, and note the normal response
- Modify one parameter or header at a time, isolating what actually changes the behavior
- Watch for status code changes, response length changes, and timing differences, not just obvious error messages
- Keep a tab per test case so responses can be compared side by side

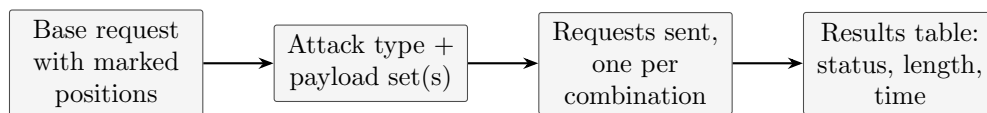
Response length is an underrated signal

A response that returns the same status code but a different content length between two payloads is often the first hint of a blind vulnerability (like blind SQL injection) where no visible error appears — Repeater's response length column makes this differences trivially easy to spot across many attempts.

# Intruder

## 6.1 The four attack types

| Attack type   | Behavior                                                                                           |
|---------------|----------------------------------------------------------------------------------------------------|
| Sniper        | One payload set, cycled through each marked position one at a time                                 |
| Battering ram | One payload set, the same value inserted into all positions simultaneously                         |
| Pitchfork     | Multiple payload sets, run in parallel (position 1 uses set 1, position 2 uses set 2, in lockstep) |
| Cluster bomb  | Multiple payload sets, every combination of values across all positions                            |



**Figure 6.1.** Every Intruder attack follows this same sequence – what differs between attack types is only how payload positions and values combine.

## 6.2 Marking payload positions

Position markers look like this in the request editor

```
username=$admin&password=test123$
```

Burp auto-suggests likely positions when a request is loaded, but manual control (via *Add \$* / *Clear \$*) is usually more precise — especially when only one specific parameter should actually be fuzzed.

## 6.3 Payload types worth knowing

| Payload type      | Use case                                                               |
|-------------------|------------------------------------------------------------------------|
| Simple list       | A fixed wordlist, e.g. common usernames or SQLi test strings           |
| Numbers           | Sequential or random numeric ranges, e.g. for ID enumeration           |
| Brute forcer      | Generates every combination of a character set up to a length          |
| Case modification | Systematically varies capitalization, useful for filter bypass testing |

**Community Edition rate limiting**

Burp Suite Community Edition throttles Intruder's request rate significantly — fine for learning the tool and small wordlists, but genuinely limiting for anything resembling a real brute-force or large fuzzing job, which is one of Professional's clearest upgrade incentives.

# Decoder and Comparer

## 7.1 Decoder

Decoder handles the constant small encoding/decoding tasks that come up during testing — URL encoding, Base64, HTML entities, hex, and hashing — without needing a separate external tool.

### Common Decoder operations

Decode as: URL  
Decode as: Base64  
Encode as: HTML  
Hash: MD5 / SHA-256

### Recursive decoding

Decoder can be chained — decode Base64, then URL-decode the result, then decode that again — which is exactly what's needed for the kind of multi-layer encoding real applications sometimes use for tokens or parameters.

## 7.2 Comparer

Comparer diffs two pieces of data — word-by-word or byte-by-byte — highlighting exactly what changed. Send two responses (or requests) to Comparer directly from any other tool to see precisely what differs between them.

### Sending to Comparer

Right-click a request/response -> "Send to Comparer"

### When Comparer earns its keep

Comparing a response before and after a single-parameter change is often far faster than manually scanning two large responses for differences — particularly useful for confirming exactly which part of a response changed when testing for subtle behavioral differences like blind injection.

# Burp Scanner

## 8.1 Passive vs. active scanning

| Mode    | Behavior                                                                       |
|---------|--------------------------------------------------------------------------------|
| Passive | Analyzes traffic that already passed through the proxy, no extra requests sent |
| Active  | Sends additional crafted requests to actively probe for vulnerabilities        |

Passive scanning is always safe to leave on

Since passive scanning never sends extra traffic, it carries essentially no risk of disrupting the target — active scanning does send extra requests and should only be pointed at hosts explicitly in scope and approved for active testing.

## 8.2 Running an active scan

Scanning a specific target

Right-click a host/URL in Target or Proxy history  
 -> "Scan"  
 Configure scan type: Crawl and audit, or Audit only

## 8.3 Reviewing scan results

Findings appear in the *Dashboard* or *Issues* view, each with a severity/confidence rating, a description, the exact request that triggered it, and (for many issue types) a working proof-of-concept — always manually verify a scanner finding in Repeater before reporting it, since automated scanners do produce false positives.

## Extensions and the BApp Store

### 9.1 What extensions add

Burp's extensibility is a major part of its ecosystem — the BApp Store (*Extender & BApp Store*) hosts community-built extensions adding everything from additional scan checks to entirely new tools within the Burp interface.

### 9.2 Notable extensions worth knowing

| Extension      | Purpose                                                               |
|----------------|-----------------------------------------------------------------------|
| Autorize       | Automated authorization/access-control testing across roles           |
| Turbo Intruder | High-performance request sending, well beyond Community's rate limits |
| J2EEScan       | Additional checks for Java/J2EE-specific vulnerabilities              |
| Logger++       | Enhanced logging and filtering across all Burp tools                  |

#### Extensions can bridge the Community/Professional gap

Some extensions (Turbo Intruder especially) meaningfully reduce Community Edition's biggest practical limitation — rate-limited request sending — making a Community-only setup considerably more capable for real testing work.

# Testing the OWASP Top 10 with Burp

## 10.1 Injection

- Repeater: manually test individual parameters with classic payloads ( ' OR '1'='1, error-based strings)
- Intruder: fuzz every parameter systematically with an injection payload list
- Watch for response length/timing differences even without visible errors (blind injection)

## 10.2 Broken authentication

- Intruder (Sniper or Cluster bomb) against login forms for credential testing
- Repeater to test session token handling: reuse, expiration, predictability
- Decoder to inspect JWT or other encoded token structures directly

## 10.3 Broken access control

- Repeater to replay a low-privilege request with a modified object ID (IDOR testing)
- Compare authenticated vs. unauthenticated access to the same endpoint
- The Authorize extension for systematic access-control testing across multiple roles

## 10.4 Security misconfiguration

- Passive scanner flags missing security headers automatically
- Proxy history review for verbose error messages leaking stack traces or internal paths
- Target site map review for exposed admin interfaces or backup files

### Burp maps to methodology, not the reverse

Burp's tools are general-purpose — the OWASP Top 10 categories are a useful checklist to make sure testing coverage is complete, not a rigid sequence to follow tool-by-tool. Real testing moves fluidly between Proxy, Repeater, and Intruder as findings emerge.

## Practical Testing Workflows

### 11.1 Workflow: testing a login form for SQL injection

#### Sequence

1. Submit a normal login, capture the request in Proxy history
2. Send to Repeater
3. Replace the username value with: `admin'--`
4. Compare the response against the baseline
5. If behavior differs, confirm with a boolean-based pair:  
`' OR '1'='1` vs. `' OR '1'='2`

### 11.2 Workflow: IDOR testing across two accounts

#### Sequence

1. Log in as User A, capture a request for User A's own resource
2. Send to Repeater
3. Change the object ID/parameter to reference User B's resource, while still authenticated as User A
4. If User B's data is returned, that's a broken access control finding

### 11.3 Workflow: directory and parameter discovery

#### Sequence

1. Browse the application normally through the proxy to seed Target
2. Send a base request to Intruder
3. Mark the path segment as a position
4. Load a directory wordlist as the payload
5. Review results sorted by status code and response length

## Detection and Defense

### 12.1 What Burp-based testing looks like from the defender's side

- A burst of requests from one source IP with systematically varied parameter values (Intruder's signature pattern)
- Requests with an unusual or default Burp User-Agent, if not deliberately spoofed
- Repeated identical requests differing only in one field, sent in rapid succession
- Requests probing for common backup files, admin paths, and verbose error triggers

### 12.2 Defensive measures

| Measure                                       | Effect                                                        |
|-----------------------------------------------|---------------------------------------------------------------|
| Rate limiting / WAF rules                     | Slows or blocks high-volume automated fuzzing patterns        |
| Generic error messages                        | Removes the verbose stack traces that guide injection testing |
| Proper access-control checks server-side      | Closes IDOR regardless of what a client requests              |
| Security headers (CSP, HSTS, X-Frame-Options) | Closes categories the passive scanner specifically flags      |

#### Logging is the real prerequisite

None of these defenses matter if nobody's watching — web server and WAF logs, correlated in a SIEM, are what actually let a defender notice active scanning or Intruder-style fuzzing in the first place, rather than only discovering an intrusion after a vulnerability was successfully exploited.